

7. Laboratory Session - ATA Disk access in PIO mode

Mark as done

In this laboratory we will program the ATA disk controller for programmed IO (PIO) access or for DMA access (optional).

ATA Disk Controller

- 2 buses on each disk controller:
 - primary bus
 - IO base: 0x1F0
 - Control base: 0x3F6
 - secondary bus
 - IO base: 0x170
 - Control base: 0x376
- 2 devices can be connected on each bus
 - the selection between the devices on the bus is done through the **Drive / Head Register**
- Important registers (offset from base)
 - Status Register (IO base +0x7)
 - **ERR** - an error occurred during execution of the previously issued command
 - **RDY** - device is **READY** to receive commands
 - **BSY** - device is preparing data for PIO transfer
 - **DRQ**
 - PIO read: data available to be read by software
 - PIO write: device is ready to accept data to write
 - **Drive / Head Register (IO base + 0x6)**
 - Selects between master and slave devices on bus
 - **Configures the addressing mode used (Bit 6)**
 - CHS Cylinder/Head/Sector
 - Requires determining the geometry of the disk through IDENTIFY DEVICE command

- Sectors start at index 1
- LBA Logical Block Addressing
 - Converting from CHS: $LBA = ((C * heads_per_cylinder + H) * sectors_per_track) + S - 1$
- Sector Count Register (IO base + 0x2)
 - Number of sectors to transfer
- Sector Number Register + Cylinder Low Register + Cylinder High Register (IO base + [0x3, 0x5])
 - Address of the first sector to be transferred
 - either in CHS format
 - either in LBA28/48
- Data Register (IO base + 0x0)
 - used to transfer data between device in PIO mode
- Device Control Register (Control base + 0x0)
 - **nIEN** - if set device does not generate interrupts
 - **HOB** - used for accessing the MSB for LBA48 commands as 2 byte writes need to be made for 16-bit registers

Offset from "I/O" base	Direction	Function	Description	Param. size LBA28/LBA48
0	R/W	Data Register	Read/Write PIO data bytes	16-bit / 16-bit
1	R	Error Register	Used to retrieve any error generated by the last ATA command executed.	8-bit / 16-bit
1	W	Features Register	Used to control command specific interface features.	8-bit / 16-bit
2	R/W	Sector Count Register	Number of sectors to read/write (0 is a special value).	8-bit / 16-bit
3	R/W	Sector Number Register (LBAlo)	This is CHS / LBA28 / LBA48 specific.	8-bit / 16-bit
4	R/W	Cylinder Low Register / (LBAmid)	Partial Disk Sector address.	8-bit / 16-bit
5	R/W	Cylinder High Register / (LBAhi)	Partial Disk Sector address.	8-bit / 16-bit

6	R/W	Drive / Head Register	Used to select a drive and/or head. Supports extra address/flag bits.	8-bit / 8-bit
7	R	Status Register	Used to read the current status.	8-bit / 8-bit
7	W	Command Register	Used to send ATA commands to the device.	8-bit / 8-bit

Offset from "Control" base	Direction	Function	Description	Param. size LBA28/LBA48
0	R	Alternate Status Register	A duplicate of the Status Register which does not affect interrupts.	8-bit / 8-bit
0	W	Device Control Register	Used to reset the bus or enable/disable interrupts.	8-bit / 8-bit
1	R	Drive Address Register	Provides drive select and head select information.	8-bit / 8-bit

Data transfer

1. Select the device through *Drive / Head Register*

2. Configure IO transfer parameters
 - *Sector Count Register*
 - *Sector Number Register + Cylinder Low Register + Cylinder High Register*

3. Write the command in the *Command register*
 - 0x20 Read command LBA28/CHS
 - 0x30 Write command LBA28/CHS

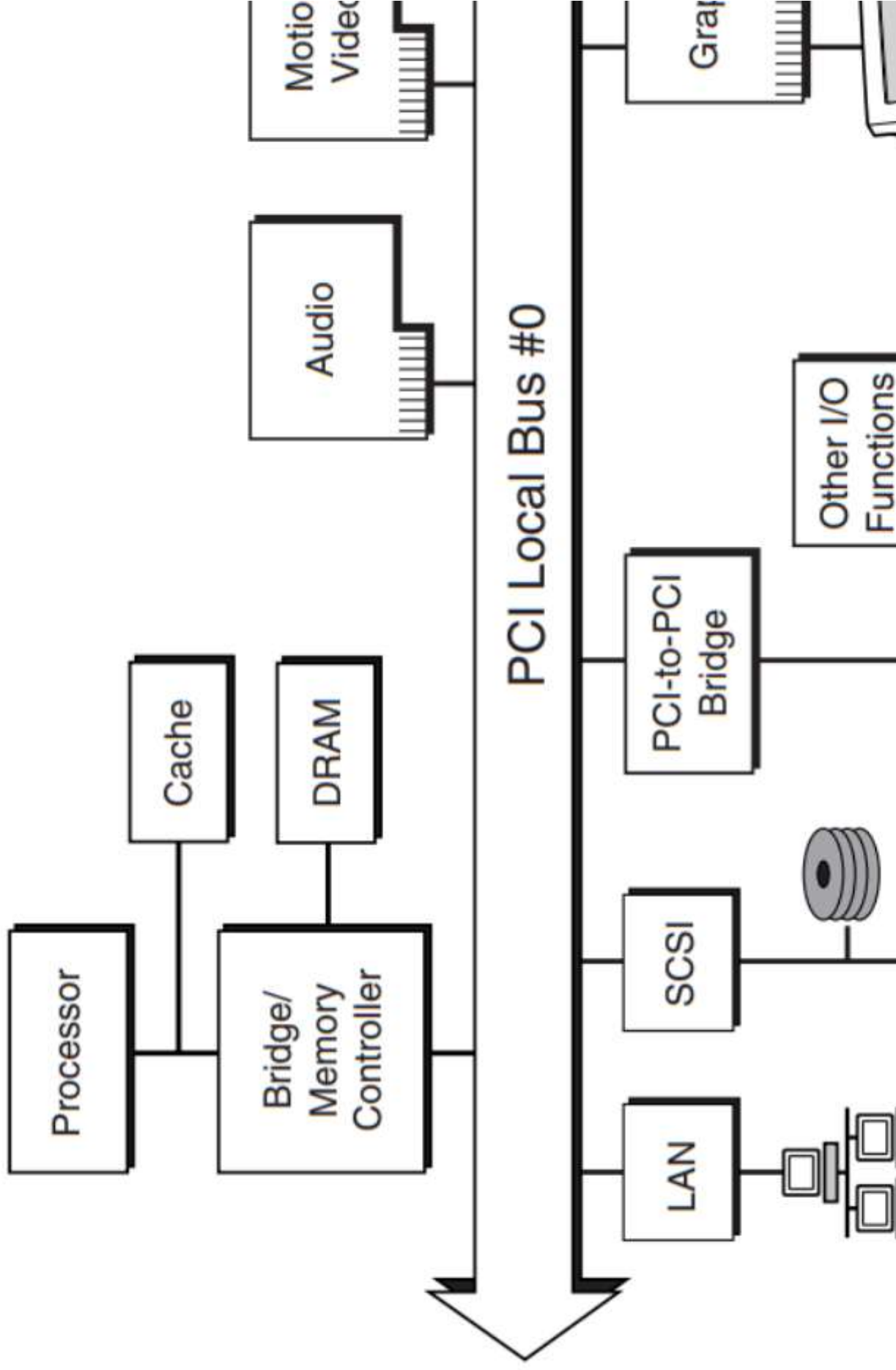
4. Transfer data through the *Data Register*
 - Access is done at the WORD granularity
 - You need to check the device is ready for the data (**DRQ**) between each access

Misc:

- Command format is detailed in [7 Command Descriptions](#)
- For interrupts which arrive at the slave PIC device (IRQ8-15) you need to transmit the EOI both on the slave and the master after you've handled the interrupt
- Basic command definitions & identify device headers *ata_commands.h* header

PCI (optional):

- Provides an industry standard method of configuring and interfacing with devices through IO and memory mapped address space
- Provides DMA access to PCI devices
- PCIe is compatible with PCI



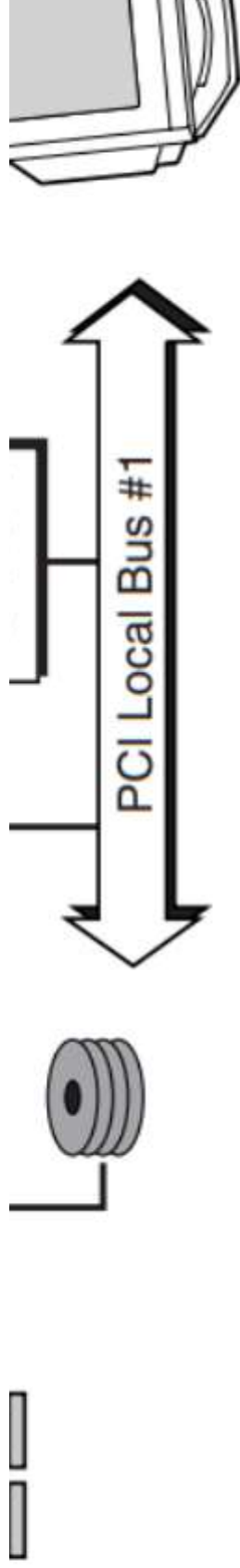


Figure 1-2: PCI System Block Diagram

Each PCI device is addressable through a bus device function address (b.d.f)

- Buses can be in the range of [0, 0xFF]
- Devices in the range of [0, 0x1F]
- Functions in the range of [0, 7]

Accessing PCI configuration space

- Write to 0xCF8 to select the b.d.f and the register number accessed (DWORD granularity) - [3.2.2.3.2 Software Generation of Configuration Transactions](#)
- Write/Read 0xCFC to transfer the data

31	16	15	0
Device ID		Vendor ID	
Status		Command	
Class Code		Revision ID	
BIST	Header Type	Latency Timer	Cacheline Size
Base Address Registers			
Cardbus CIS Pointer			

Subsystem ID		Subsystem Vendor ID	2Ch
Expansion ROM Base Address			30h
Reserved		Capabilities Pointer	34h
Reserved			38h
Max_Lat	Min_Gnt	Interrupt Pin	Interrupt Line
			3Ch

A-0191

Figure 6-1: Type 00h Configuration Space Header

To determine if a PCI device is present, scan each bus/device/function and if **Vendor ID** is 0xFFFF there is no device present - [6.1 Configuration Space Organization](#)

- PCI configuration space may be up to 256 bytes and can only be accessed through IO ports
- PCIe configuration space may be up to 4096 bytes and can be access either through MMIO or through IO ports for the first 256B

Base Address Registers (BARs) provide the address of device specific memory either through IO or MMIO access

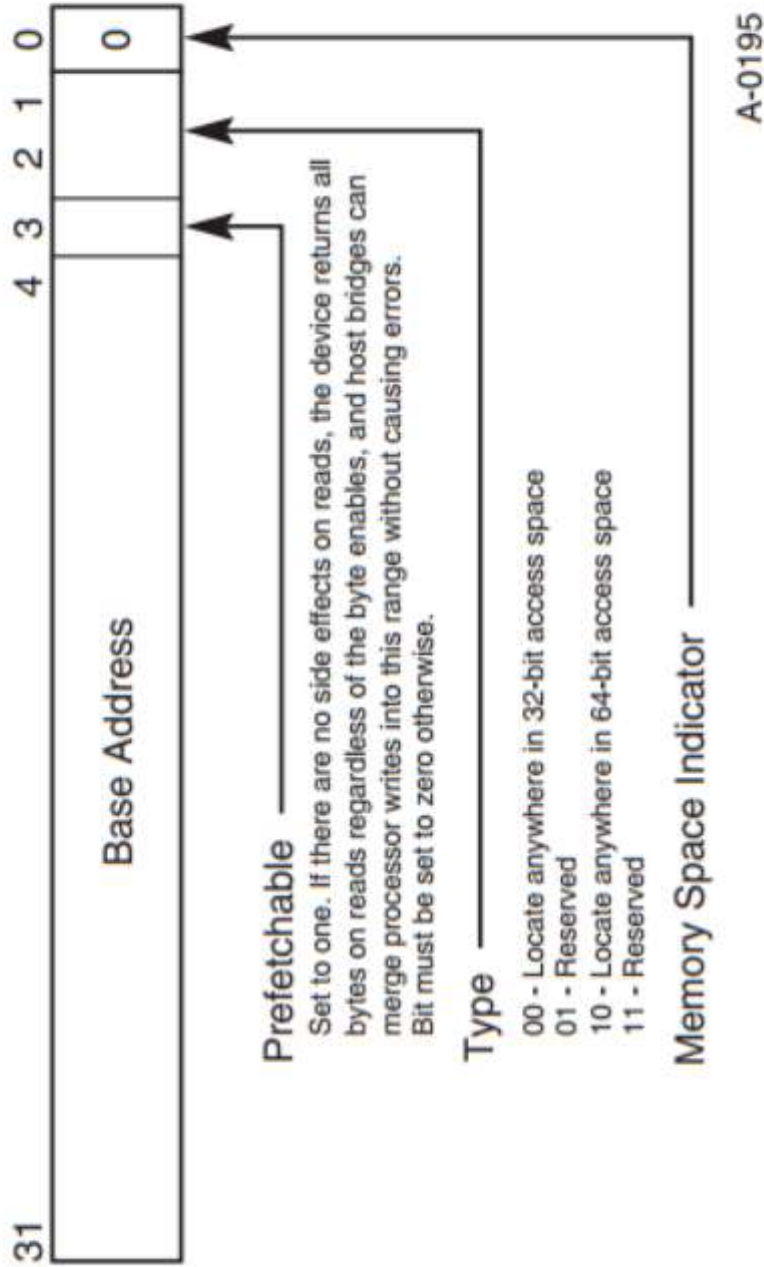


Figure 6-5: Base Address Register for Memory



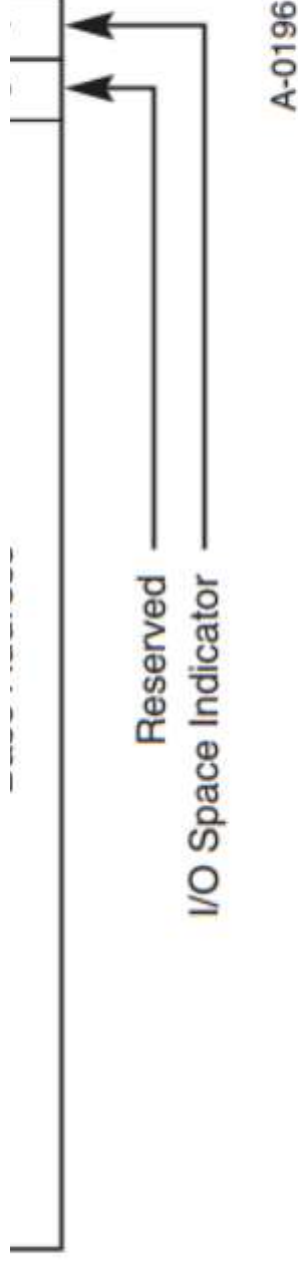


Figure 6-6: Base Address Register for I/O

Assignment:

- Add support for reading data from the disk
 - Determine the presence of ATA devices on the controller using the `IDENTIFY DEVICE` (`0xEC`)
 - If after the command *Status Register* is 0=> there is no device present
 - If present you need to read *Status Register* until **BSY** becomes ZERO
 - If **ERR** is set before **BSY** becomes zero => ATAPI connected => optical disk => ignore it
 - Check for the presence on both buses and both devices on each bus => 2x2 loops
 - Data transfer should be done through PIO
 - (Optional) Data transfer is done through DMA
 - (Optional) Scan the PCI bus to determine presence of ATA disk controllers and determine IO base addresses from BARs
 - `ClassCode: 0x1 - MassStorageController`
 - `SubClass 0x1 - MassStorageIDE`
 - For BAR format see [Table 11 - Table 15](#)
- Add a new command **printmbr** in the console which displays the contents of the first disk sector in a similar format to hex editors:

[illegible]

Deadline: 23.11 23:55

1. Create a copy of floppy.img from the *bin* folder and name it *disk.img*

ata0-master: type=none #!update! e.g.: disk, path=".\\bin\\hdd.img", mode=flat

ata0-master: type=disk, path=".\\bin\\disk.img", mode=flat, cylinders=2, heads=16, spt=63, model="Generic 1234", biosdetect=none, translation=lba

- ATA

- https://wiki.osdev.org/ATA_PIO_Mode
- https://wiki.osdev.org/PCI_IDE_Controller
- <https://forum.osdev.org/viewtopic.php?f=1&p=167798>
- [ATA & ATAPI specification](#)
- (Optional) PCI
 - <https://wiki.osdev.org/PCI>
 - [PCI Local Bus Specification V3.0 6. Configuration Space](#)
- (Optional) DMA
 - [Bus Master IDE Specification](#)
 - [IDE Controller programmer reference](#)
 - 4.1.1 PCI Configuration Registers obtain BAR4 Bus Master IDE Command and Status Register

<https://moodle.cs.utcluj.ro/mod/page/view.php?id=59738>

